

第十二章 查询处理

查询处理：从数据库中提取数据时涉及的一系列活动

12.1 概述

查询处理步骤如图所示，基本步骤包括：语法分析与翻译、优化、执行

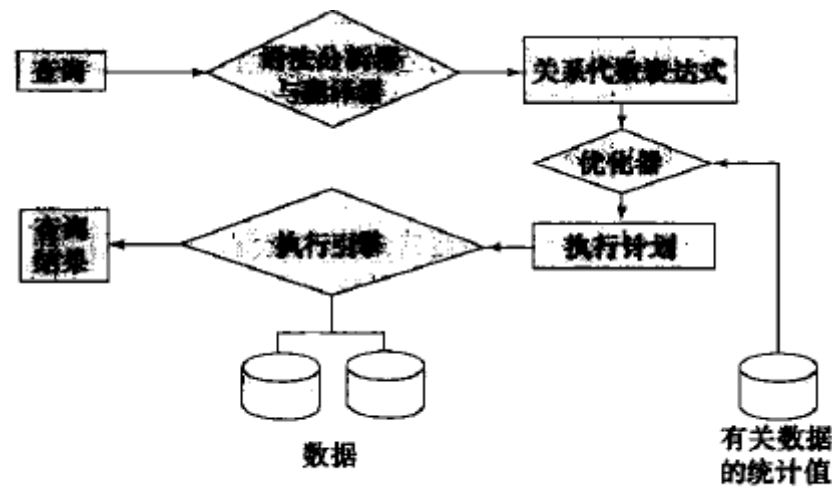


图 12-1 查询处理的步骤

查询处理开始之前，系统必须将查询语句翻译成可使用的形式

给定一个查询，一般会有多种计算结果的方法

example:

```
select salary
from instructor
where salary < 75000;
```

该查询语句可翻译成下面两个关系代数表达式中的任意一个：

- $\sigma_{salary < 75000} (\Pi_{salary} (instructor))$
- $\Pi_{salary} (\sigma_{salary < 75000} (instructor))$

计算原语：加了“如何执行”注释的关系代数运算

查询执行计划：执行一个查询的愿与操作序列

查询执行引擎：接受一个查询执行计划，执行该计划并把结果返回给查询

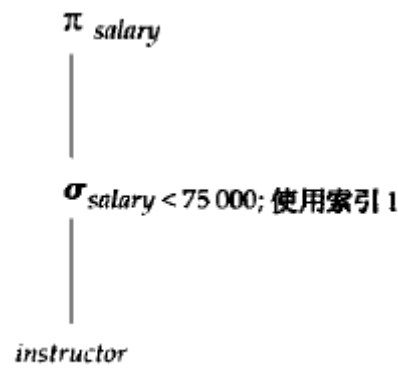


图 12-2 一个查询执行计划

12.2 查询代价的度量

在数据库系统中，最主要的代价是在磁盘上存取数据的代价

传送磁盘块数以及搜索磁盘次数被用来度量查询计算机化的代价

- 传输一个块的数据平均消耗 t_T 秒
- 磁盘块平均访问时间 t_S 秒

显然一次传输 b 块且执行 S 次磁盘搜索将消耗 $b * t_T + S * t_S$ 秒

一个查询计算计划**响应时间**就是所有的这些开销，并且可以作为计划的代价的度量

12.3 选择运算

在查询处理中，**文件扫描**是存取数据最低级的操作

12.3.1 使用文件扫描和索引的选择

- **A1 (线性搜索)**：系统扫描每一个文件块，对所有记录都进行测试

使用索引的搜索算法称为**索引扫描**

- **A2 (主索引，码属性等值比较)**：使用索引检索到满足相应等值条件的唯一一条记录
- **A3 (主索引，非码属性等值比较)**：利用主索引检索到多条记录，与A2相比需要取多条记录
- **A4 (辅助索引，等值比较)**：

若等值条件是码属性上的，则该策略可检索到唯一一条记录

这种情况下代价与A2相似

若索引字段是非码属性的，则可能检索到多条记录

这种情况下每检索一条记录都需要一次I/O操作（搜索+磁盘块传输）

- (A5, A6不重要不管)

	算 法	开 销	原 因
A1	线性搜索	$t_i + b_i * t_T$	一次初始搜索加上 b_i 个块传输， b_i 表示在文件中的块数量
A1	线性搜索，码属性等值比较	平均情形 $t_i + (b_i/2) * t_T$	因为最多一条记录满足条件，所以只要找到所需的记录，扫描就可以终止。在最坏的情形下，仍需要 b_i 个块传输
A2	B* 树主索引，码属性等值比较	$(h_i + 1) * (t_T + t_i)$	(其中 h_i 表示索引的高度)。索引查找穿越树的高度，再加上一次 I/O 来取记录；每个这样的 I/O 操作需要一次搜索和一次块传输
A3	B* 树主索引，非码属性等值比较	$h_i * (t_T + t_i) + b_i * t_T$	树的每层一次搜索，第一个块一次搜索。 b_i 是包含具有指定搜索码记录的块数。假定这些块是顺序存储(因为是主索引)的叶子块并且不需要额外搜索
A4	B* 树辅助索引，码属性等值比较	$(h_i + 1) * (t_T + t_i)$	这种情形和主索引相似
A4	B* 树辅助索引，非码属性等值比较	$(h_i + n) * (t_T + t_i)$	(其中 n 是所取记录数。)索引查找的代价和 A3 相似，但是每条记录可能在不同的块上，这需要每条记录一次搜索。如果 n 值比较大，代价可能会非常高
A5	B* 树主索引，比较	$h_i * (t_T + t_i) + b_i * t_T$	和 A3，非码属性等值比较情形一样
A6	B* 树辅助索引，比较	$(h_i + n) * (t_T + t_i)$	和 A4，非码属性等值比较情形一样

图 12-3 选择算法代价估计

12.4 排序

数据排序有重要作用，有两个原因：

1. SQL查询会指明对结果进行排序
2. 当输入的关系已经排序时，关系运算中的一些运算（如连接运算）能够得到高效实现

12.4.1 外部排序归并算法

外排序：不能全部放在内存中的关系的排序

M表示内存缓冲区中可以用于排序的块数，即内存的缓冲区能容纳的磁盘块数

1. 第一阶段，建立多个排好序的归并段，每个归并段都是排序过的，但仅包含关系中的部分记录

```

i = 0;
repeat
    读入关系的 M 块数据或剩下的不足 M 块的数据;
    在内存中对关系的这一部分进行排序;
    将排好序的数据写到归并段文件 Ri 中;
    i = i + 1;
until 到达关系末尾
    
```

2. 第二阶段，对归并段进行归并。假定归并段总数 $N < M$ ，为 N 个归并段文件 R_i 各分配一个内存缓冲块，并分别读入一个数据块；

```

repeat
    在所有缓冲块中按序挑选第一个元组;
    将该元组作为输出写出，并将其从缓冲块中删除;
    if 任何一个归并段文件 Ri 的缓冲块为空并且没有到达 Ri 末尾
        then 读入 Ri 的下一块到相应的缓冲块
until 所有的缓冲块均空
    
```

该算法对 N 个归并段进行归并，因此它被称为 N 路归并

一般而言，关系比内存大得多，即 $N \gg M$ ，因此归并要分多趟进行，每趟可以用 $M-1$ 个归并段作为输入。这样每趟下来，归并段的数目都减少到原来的 $1/(M-1)$ ，直至归并段数目小于 M 得到排序的输出结果

example: $M=3$ (两块用于输入，一块用于输出)

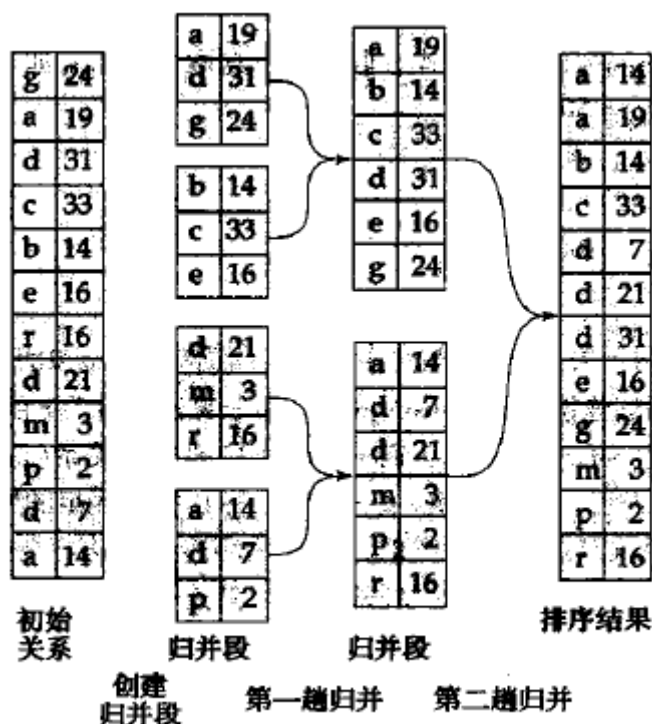


图 12-4 使用归并排序的外排序

12.4.2 外部排序归并的代价分析

b_r 代表包含关系 r 中记录的磁盘块数

磁盘块传输：

第一阶段：

- 磁盘块传输： $2b_r$ 次

第二阶段:

- 归并趟数: $\lceil \log_{M-1}(b_r/M) \rceil$
- 每一趟归并读写各一次
- 最后一趟归并可以只产生结果而不写入磁盘

磁盘块传输总数: $2b_r + b_r(2 * \lceil \log_{M-1}(b_r/M) \rceil - 1) = b_r(2\lceil \log_{M-1}(b_r/M) \rceil + 1)$

磁盘搜索:

第一阶段:

- 读取写回归并段: $2\lceil b_r/M \rceil$

第二阶段:

- 每次从一个归并段读取 b_b 块数据, 每趟归并需要作 $\lceil b_r/b_b \rceil$ 次磁盘搜索以读取数据
- 假设输出阶段也分配了 b_b 个块, 每一趟可以归并 $\lfloor M/b_b \rfloor - 1$ 个归并段

磁盘搜索总次数: $2\lceil b_r/M \rceil + \lceil b_r/b_b \rceil(2\lceil \log_{\lfloor M/b_b \rfloor - 1}(b_r/M) \rceil - 1)$

(本质上就是第二阶段的M和 b_r 都除以 b_b)

12.5 连接运算

等值连接: 表示形如 $r \bowtie_{r.A=s.B} s$ 的连接, 其中A,B分别为r与s的属性或属性组

- student记录数: $n_{student} = 5000$
- student磁盘块数: $b_{student} = 100$
- takes记录数: $n_{takes} = 10000$
- takes磁盘块数: $b_{takes} = 400$

12.5.1 嵌套循环连接

该算法主要由两个嵌套的for循环构成, 因此被称为**嵌套循环连接**

```
for each 元组  $t_r$  in  $r$  do begin
    for each 元组  $t_s$  in  $s$  do begin
        测试元组对 $(t_r, t_s)$ 是否满足连接条件  $\theta$ 
        如果满足, 把  $t_r \cdot t_s$  加到结果中
    end
end
```

图 12-5 嵌套循环连接

对于 $r \bowtie s$, r 被称为连接的**外层关系**, s 被称为连接的**内层关系**

- r, s 中元组数: n_r, n_s
- 包含关系 r, s 中元组的磁盘块数: b_r, b_s
- 需要考虑的元组对数目: $n_r * n_s$
- 对关系 r 中的每一条记录, 我们必须对 s 作一次完整的扫描

	最坏情况	最好情况
块传输	$n_r * b_s + b_r$	$b_r + b_s$
磁盘搜索	$n_r + b_r$	2

12.5.2 块嵌套循环连接

```

for each 块  $B_r$  of  $r$  do begin
    for each 块  $B_s$  of  $s$  do begin
        for each 元组  $t_r$  in  $B_r$  do begin
            for each 元组  $t_s$  in  $B_s$  do begin
                测试元组对  $(t_r, t_s)$  是否满足连接条件
                如果满足, 把  $t_r \cdot t_s$  加到结果中
            end
        end
    end
end

```

图 12-6 块嵌套循环连接

块嵌套循环连接是嵌套循环连接的一个变种，内层关系的每一块与外层关系的每一块形成一对

	最坏情况	最好情况
块传输	$b_r * b_s + b_r$	$b_r + b_s$
磁盘搜索	$2b_r$	2

嵌套循环与块嵌套循环算法的性能可以进一步地改进：

- 如果自然连接或等值连接中的连接属性是内层关系的码，则对每个外层关系元组，内层循环一旦找到了首条匹配元组就可以终止。
- 在块嵌套循环连接算法中，外层关系可以不用磁盘块作为分块的单位，而以内存中最多能容纳的大小为单位，当然同时要留出足够的缓冲空间给内层关系及输出结果使用。也就是说，如果内存有 M 块，我们一次读取外层关系中的 $M - 2$ 块，当我们读取到内层关系中的每一块时，我们把它与外层关系中的所有 $M - 2$ 块做连接。这种改进使内层关系的扫描次数从 b_s 次减少到 $\lceil b_s / (M - 2) \rceil$ 次，这里的 b_s 是外层关系所占的块数。这样全部代价为 $\lceil b_s / (M - 2) \rceil * b_r + b_r$ 次块传输和 $2 \lceil b_s / (M - 2) \rceil$ 次磁盘搜索。
- 对内层循环轮流做向前、向后的扫描。该扫描方法对磁盘块读写请求进行排序，使得上一次扫描时留在缓冲区中的数据可以重用，从而减少磁盘存取次数。
- 若内层循环的连接属性上有索引，可以用更有效的索引查找法替代文件扫描法。这一改进在 12.5.3 节讲述。

12.5.3 索引嵌套循环连接

(不管了)

12.6 其他运算

12.6.1 去除重复

用排序方法可以很容易地实现去除重复

12.6.2 投影

对每个元组作投影，所得结果关系可能有重复记录，然后去除重复记录

12.7 表达式计算

12.7.1 物化

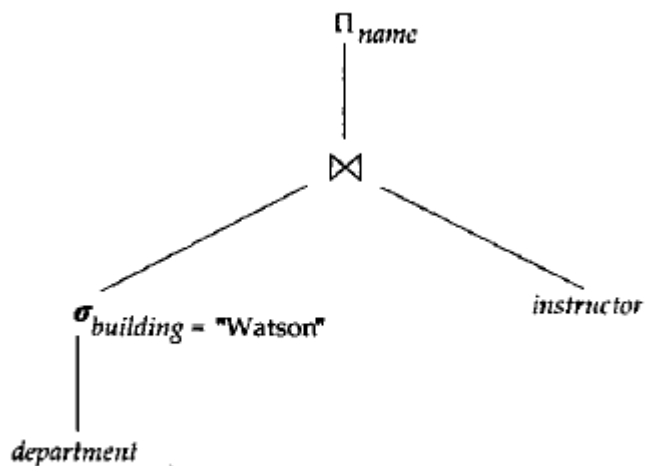


图 12-11 一个表达式的图形化表示

$\Pi_{name}(\sigma_{building = "Watson"}(department) \bowtie instructor)$

运算符数：对表达式做图形化表示

当采用物化方法时，从表达式的最底层的运算（在树的底部）开始

物化计算：运算的每个中间结果被创建（物化），然后用于下一层的运算

12.7.2 流水线

流水线计算：将多个关系操作组合成一个操作的流水线来实现，其中一个操作的结果将传送到下一个操作

优势：

1. 消除读和写临时关系的代价，减少查询计算代价
2. 如果一个查询计算计划的根操作符及其输入合并到流水线中，那么可以迅速开始产生查询结果

12.7.2.1 流水线的实现

1. *需求驱动的流水线*，系统不停地向位于流水线顶端的操作发出需要元组的请求
2. *生产者驱动的流水线*，各操作不等待元组请求，而是积极地产生元组

12.8 总结

见P325

第十三章 查询优化

查询优化：从给定查询的多种可能策略中找出最有效的查询执行计划的一种处理过程

13.1 概述

example:

$\Pi_{name, title}((\sigma_{dept_name = "Music"}(instructor)) \bowtie (teaches \bowtie \Pi_{course_id, title}(course)))$

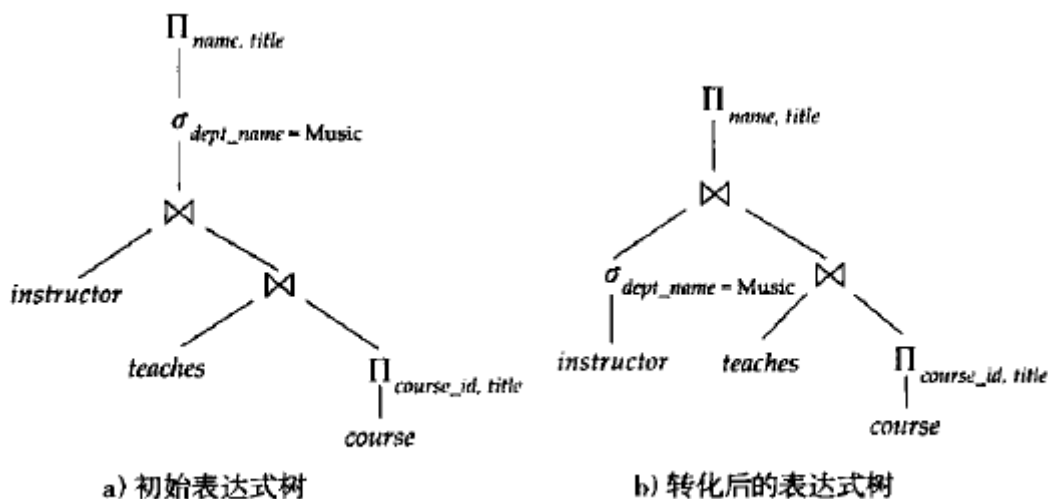


图 13-1 等价表达式

给定一个关系代数表达式，查询优化器的任务是产生一个查询执行计划，该计划能获得与原关系表达式相同的结果，并且得到结果集的执行代价最小

查询执行计划的产生步骤：

1. 产生逻辑上与给定表达式等价的表达式
2. 对所产生的表达式以不同方式作注释，产生不同的查询计划
3. 估计每个执行计划的代价，选择代价最小的一个

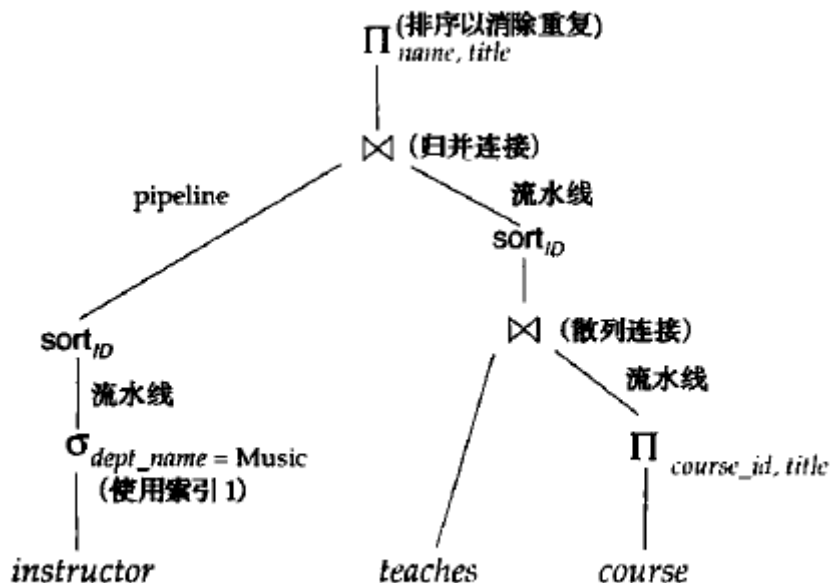


图 13-2 一个执行计划

13.2 关系表达式的转换

如果两个关系表达式在每一个有效数据库实例中都会产生相同的元组集，则我们称它们是等价的

13.2.1 等价规则

等价规则指出两种不同形式的表达式是等价的

1. $\sigma_{\theta_1 \wedge \theta_2}(E) = \sigma_{\theta_1}(\sigma_{\theta_2}(E))$
2. $\sigma_{\theta_1}(\sigma_{\theta_2}(E)) = \sigma_{\theta_2}(\sigma_{\theta_1}(E))$

$$3. \Pi_{L_1}(\Pi_{L_2}(\dots(\Pi_{L_n}(E))\dots)) = \Pi_{L_1}(E)$$

$$4. \text{ a. } \sigma_{\theta}(E_1 \bowtie E_2) = E_1 \bowtie_{\theta} E_2$$

$$\text{ b. } \sigma_{\theta_1}(E_1 \bowtie_{\theta_2} E_2) = E_1 \bowtie_{\theta_1 \wedge \theta_2} E_2$$

$$5. E_1 \bowtie_{\theta} E_2 = E_2 \bowtie_{\theta} E_1$$

$$6. (E_1 \bowtie E_2) \bowtie E_3 = E_1 \bowtie (E_2 \bowtie E_3)$$

$$(E_1 \bowtie_{\theta_1} E_2) \bowtie_{\theta_2 \wedge \theta_3} E_3 = E_1 \bowtie_{\theta_1 \wedge \theta_3} (E_2 \bowtie_{\theta_2} E_3)$$

where θ_2 involves attributes from only E_2 and E_3 .

$$7. \sigma_{\theta_0}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta_0}(E_1)) \bowtie_{\theta} E_2$$

$$\sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta_1}(E_1)) \bowtie_{\theta} (\sigma_{\theta_2}(E_2))$$

$$8. \Pi_{L_1 \cup L_2}(E_1 \bowtie_{\theta} E_2) = (\Pi_{L_1}(E_1)) \bowtie_{\theta} (\Pi_{L_2}(E_2))$$

$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_{\theta} E_2) = \Pi_{L_1 \cup L_2}((\Pi_{L_1 \cup L_3}(E_1)) \bowtie_{\theta} (\Pi_{L_2 \cup L_4}(E_2)))$$

$$9. E_1 \cup E_2 = E_2 \cup E_1$$

$$E_1 \cap E_2 = E_2 \cap E_1$$

$$10. (E_1 \cup E_2) \cup E_3 = E_1 \cup (E_2 \cup E_3)$$

$$(E_1 \cap E_2) \cap E_3 = E_1 \cap (E_2 \cap E_3)$$

$$11. \sigma_{\theta}(E_1 - E_2) = \sigma_{\theta}(E_1) - \sigma_{\theta}(E_2)$$

$$\sigma_{\theta}(E_1 - E_2) = \sigma_{\theta}(E_1) - E_2$$

$$12. \Pi_L(E_1 \cup E_2) = (\Pi_L(E_1)) \cup (\Pi_L(E_2))$$

1. 合取选择运算可分解为单个选择运算的序列。该变换称为 σ 的级联：

$$\sigma_{\theta_1 \wedge \theta_2}(E) = \sigma_{\theta_1}(\sigma_{\theta_2}(E))$$

2. 选择运算满足交换律(commutative)：

$$\sigma_{\theta}(\sigma_{\theta_1}(E)) = \sigma_{\theta_1}(\sigma_{\theta}(E))$$

3. 一系列投影运算中只有最后一个运算是必需的, 其余的可省略。该转换也可称为 Π 的级联:

$$\Pi_{L_1}(\Pi_{L_2}(\dots(\Pi_{L_n}(E))\dots)) = \Pi_{L_n}(E)$$

4. 选择操作可与笛卡儿积以及 θ 连接相结合:

$$a. \sigma_{\theta}(E_1 \times E_2) = E_1 \bowtie_{\theta} E_2.$$

该表达式就是 θ 连接的定义。

$$b. \sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie_{\theta_1} E_2) = E_1 \bowtie_{\theta_1 \wedge \theta_2} E_2$$

5. θ 连接运算满足交换律:

$$E_1 \bowtie_{\theta} E_2 = E_2 \bowtie_{\theta} E_1$$

事实上, 左端和右端的属性顺序不同, 所以如果考虑属性顺序的话, 等式不成立。可以对等价规则的其中一端加入一个投影操作以适当重排属性, 但为了简化, 我们省略了该投影并且在大部分例子中忽略属性顺序。

回忆一下, 自然连接是 θ 连接的特例, 因此自然连接也满足交换律。

6. a. 自然连接运算满足结合律(associative):

$$(E_1 \bowtie E_2) \bowtie E_3 = E_1 \bowtie (E_2 \bowtie E_3)$$

- b. θ 连接具有以下方式的结合律:

$$(E_1 \bowtie_{\theta_1} E_2) \bowtie_{\theta_2 \wedge \theta_3} E_3 = E_1 \bowtie_{\theta_1 \wedge \theta_2} (E_2 \bowtie_{\theta_3} E_3)$$

其中 θ_2 只涉及 E_2 与 E_3 的属性。由于其中的任意一个条件都可为空, 因此笛卡儿积($\times$)运算也满足结合律。连接运算满足结合律、交换律在查询优化中对于重排连接顺序是很重要的。

7. 选择运算在下面两个条件下对 θ 连接运算具有分配律:

- a. 当选择条件 θ_0 中的所有属性只涉及参与连接运算的表达式之一(比如 E_1)时, 满足分配律:

$$\sigma_{\theta_0}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta_0}(E_1)) \bowtie_{\theta} E_2$$

- b. 当选择条件 θ_1 只涉及 E_1 的属性, 选择条件 θ_2 只涉及 E_2 的属性时, 满足分配律:

$$\sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta_1}(E_1)) \bowtie_{\theta} (\sigma_{\theta_2}(E_2))$$

8. 投影运算在下面条件下对 θ 连接运算具有分配律:

- a. 令 L_1 、 L_2 分别代表 E_1 、 E_2 的属性。假设连接条件 θ 只涉及 $L_1 \cup L_2$ 中的属性, 则:

$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_{\theta} E_2) = (\Pi_{L_1}(E_1)) \bowtie_{\theta} (\Pi_{L_2}(E_2))$$

- b. 考虑连接 $E_1 \bowtie_{\theta} E_2$ 。令 L_1 、 L_2 分别代表 E_1 、 E_2 的属性集; 令 L_3 是 E_1 中出现在连接条件 θ 中但不在 $L_1 \cup L_2$ 中的属性; 令 L_4 是 E_2 中出现在连接条件 θ 中但不在 $L_1 \cup L_2$ 中的属性。那么:

$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_{\theta} E_2) = \Pi_{L_1 \cup L_2}((\Pi_{L_1 \cup L_3}(E_1)) \bowtie_{\theta} (\Pi_{L_2 \cup L_4}(E_2)))$$

9. 集合的并与交满足交换律。

$$E_1 \cup E_2 = E_2 \cup E_1$$

$$E_1 \cap E_2 = E_2 \cap E_1$$

集合的差运算不满足交换律。

10. 集合的并与交满足结合律。

$$(E_1 \cup E_2) \cup E_3 = E_1 \cup (E_2 \cup E_3)$$

$$(E_1 \cap E_2) \cap E_3 = E_1 \cap (E_2 \cap E_3)$$

11. 选择运算对并、交、差运算具有分配律:

$$\sigma_r(E_1 - E_2) = \sigma_r(E_1) - \sigma_r(E_2)$$

类似地, 上述等价规则将“-”替换成 $\cup$ 或 $\cap$ 时也成立。进一步有:

$$\sigma_r(E_1 - E_2) = \sigma_r(E_1) - E_2$$

上述等价规则将“-”替换成 $\cap$ 时也成立, 但将“-”替换成 $\cup$ 时不成立。

12. 投影运算对并运算具有分配律。

$$\Pi_L(E_1 \cup E_2) = (\Pi_L(E_1)) \cup (\Pi_L(E_2))$$

见P331-332

总的来说就是不同形式的交换律、结合律、分配律

13.2.2 转换的例子

下面举例说明等价规则的用法。我们用大学这个例子，其关系模式如下：

Instructor(*ID*, *name*, *dept_name*, *salary*)
teaches(*ID*, *course_id*, *sec_id*, *semester*, *year*)
course(*course_id*, *title*, *dept_name*, *credits*)

在 13.1 节的例子中，表达式：

$$\Pi_{name, title}(\sigma_{dept_name = \text{"Music"}}(instructor \bowtie (teaches \bowtie \Pi_{course_id, title}(course))))$$

转换成以下表达式：

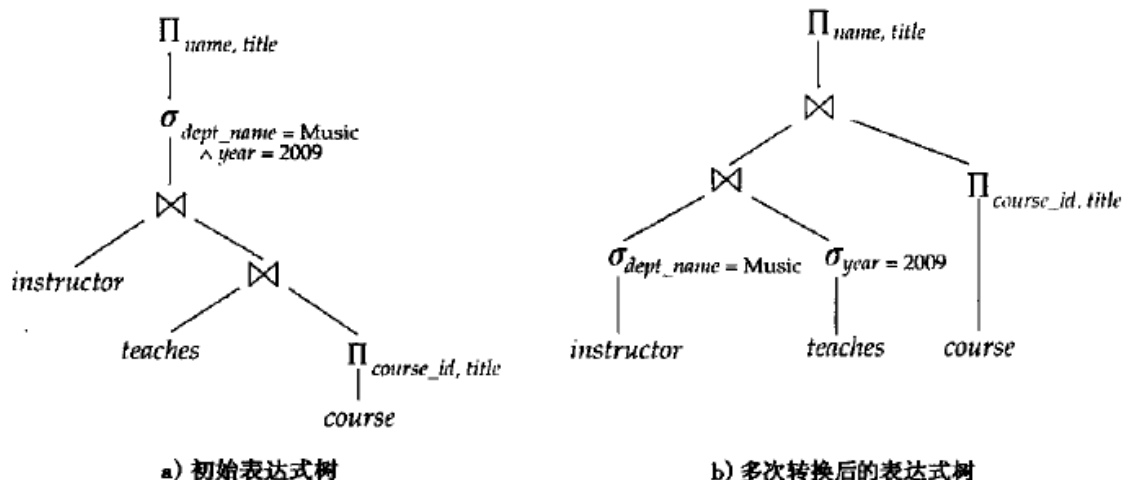
$$\Pi_{name, title}((\sigma_{dept_name = \text{"Music"}}(instructor)) \bowtie (teaches \bowtie \Pi_{course_id, title}(course)))$$


图 13-4 多次转换

见P333

13.2.3 连接的次序

一个好的连接运算次序对于减少临时结果的大小是很重要的

$(r_1 \bowtie r_2) \bowtie r_3 = r_1 \bowtie (r_2 \bowtie r_3)$ 虽然这两个表达式等价，但是计算它们的代价可能不同

13.2.4 等价表达式的枚举

```

procedure genAllEquivalent(E)
  EQ = { E }
  repeat
    将 EQ 中的每条表达式 Ei 与每条等价规则 Rj 进行匹配
    如果 Ei 的某个子表达式与 Rj 的某一边相匹配
      生成一个与 Ei 等价的 E'，其中 ei 被替换成 Rj 的另一边
      如果 E' 不在 EQ 中，则将其添加到 EQ 中
  until 不再有新的表达式可以添加到 EQ 中
  
```

图 13-5 产生所有等价表达式的过程

13.3 表达式结果集统计大小的估计

13.3.1 目录信息

- n_r : 关系 r 的元组数
- b_r : 包含关系 r 中元组的磁盘块数

- l_r : 关系 r 中每个元组的字节数
- f_r : 关系 r 的块因子——一个磁盘块能容纳关系 r 中元组的个数
- $V(A, r)$: 关系 r 中属性 A 中出现的非重复值个数

显然有等式 $b_r = \lceil n_r / f_r \rceil$

等宽直方图: 取值范围分成相等大小的区间

等深直方图: 每个区间的取值个数相同

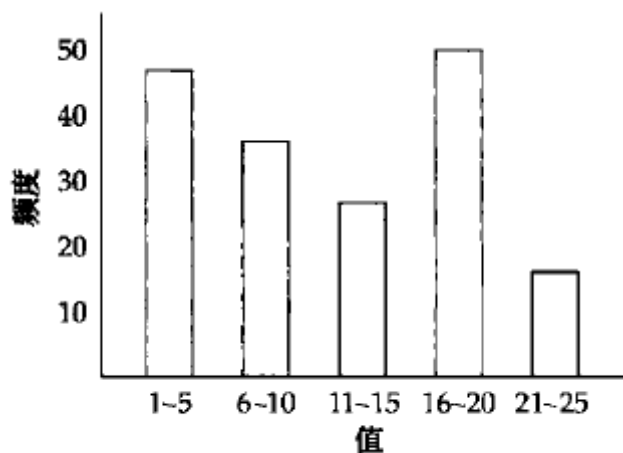


图 13-6 直方图示例

13.3.2 选择运算结果大小的估计

- $\sigma_{A=a}(r)$: $n_r / V(A, r)$ 个元组
- $\sigma_{A \leq a}(r)$: $n_r \cdot \frac{v - \min(A, r)}{\max(A, r) - \min(A, r)}$ 个元组
- 复杂选择:
 - 合取: $\sigma_{\theta_1 \wedge \theta_2 \wedge \dots \wedge \theta_n}(r)$.

关系中的一个元组满足条件选择 θ_i 的概率为 s_i / n_r , 这概率被称为选择 $\sigma_{\theta_i}(r)$ 的**中选率**

$$\text{合取的元组数量: } n_r * \frac{s_1 * s_2 * \dots * s_n}{n_r^n}$$

- 析取: $\sigma_{\theta_1 \vee \theta_2 \vee \dots \vee \theta_n}(r)$.

析取的元组数量:

$$n_r * \left(1 - \left(1 - \frac{s_1}{n_r} \right) * \left(1 - \frac{s_2}{n_r} \right) * \dots * \left(1 - \frac{s_n}{n_r} \right) \right)$$

- 取反: $\sigma_{\neg \theta}(r)$

取反的元组数量: $n_r - \sigma_{\theta}(r)$

13.3.3 连接运算结果大小的估计

对于 $r \bowtie s$, 令 $r(R)$ 和 $s(S)$ 为两个关系

- 若 $R \cap S = \varnothing$, 则两关系没有共同属性, 此时 $r \bowtie s = r \times s$
- 若 $R \cap S$ 是 R 或 S 的码, 则此时得到元组数与 r 或 s 中的元组数相同
- 若 $R \cap S$ 既不是 R 也不是 S 的码, 假定 $R \cap S = \{A\}$, 此时应该有 $(n_r \times n_s) / V(A, s)$ 个元组

13.3.4 其它运算的结果集大小的估计

- **投影**: 估计为 $V(A, r)$, 因为投影去除了重复元组
- **聚集**:
 $\sum_A G_F(r)$ 的大小是 $V(r)$, 因为对 A 的任意一个不同取值在 $\sum_A G_F(r)$ 中有一个元组与其对应。
- **集合运算**: 可以重写为合取、析取以及取反进行计算
- **外连接**:
外连接: $r \Join s$ 的大小估计为 $r \bowtie s$ 的大小加上关系 r 的大小; 对 $r \Join s$ 的估计与对 $r \bowtie s$ 的估计是对称的, 而 $r \Join s$ 的估计为 $r \bowtie s$ 的大小加上关系 r 和关系 s 的大小。以上三种估计可能不精确, 但提供了结果集大小的上界。

13.4 执行计划选择

基于代价的优化器从给定查询等价的所有查询执行计划空间中进行搜索, 并选择估计代价最小的一个

13.4.1 基于代价的连接顺序选择

考虑表达式 $r_1 \bowtie r_2 \bowtie \dots \bowtie r_n$, 当 $n=3$ 时, 有12种不同的连接顺序

$$\begin{array}{cccc} r_1 \bowtie (r_2 \bowtie r_3) & r_1 \bowtie (r_3 \bowtie r_2) & (r_2 \bowtie r_3) \bowtie r_1 & (r_3 \bowtie r_2) \bowtie r_1 \\ r_2 \bowtie (r_1 \bowtie r_3) & r_2 \bowtie (r_3 \bowtie r_1) & (r_1 \bowtie r_3) \bowtie r_2 & (r_3 \bowtie r_1) \bowtie r_2 \\ r_3 \bowtie (r_1 \bowtie r_2) & r_3 \bowtie (r_2 \bowtie r_1) & (r_1 \bowtie r_2) \bowtie r_3 & (r_2 \bowtie r_1) \bowtie r_3 \end{array}$$

对于 $(r_1 \bowtie r_2 \bowtie r_3) \bowtie r_4 \bowtie r_5$

们需要检查 144 种连接顺序。然而, 一旦我们给关系子集 $\{r_1, r_2, r_3\}$ 找到了最佳连接顺序, 我们可以用此顺序进一步与 r_4, r_5 进行连接, 舍弃 $r_1 \bowtie r_2 \bowtie r_3$ 中其他代价较大的连接顺序。这样, 我们不必一一检查 144 种连接顺序, 而只须检查 $12 + 12$ 种顺序。

13.4.3 启发式优化

查询优化器使用**启发式方法**来减少优化代价

对关系代数查询进行转换的例子:

- 尽早执行选择运算
- 尽早执行投影运算

左深连接顺序用于流水线计算特别方便, 因为右操作对象是一个已存储的关系, 每个连接只有一个输入来自流水线

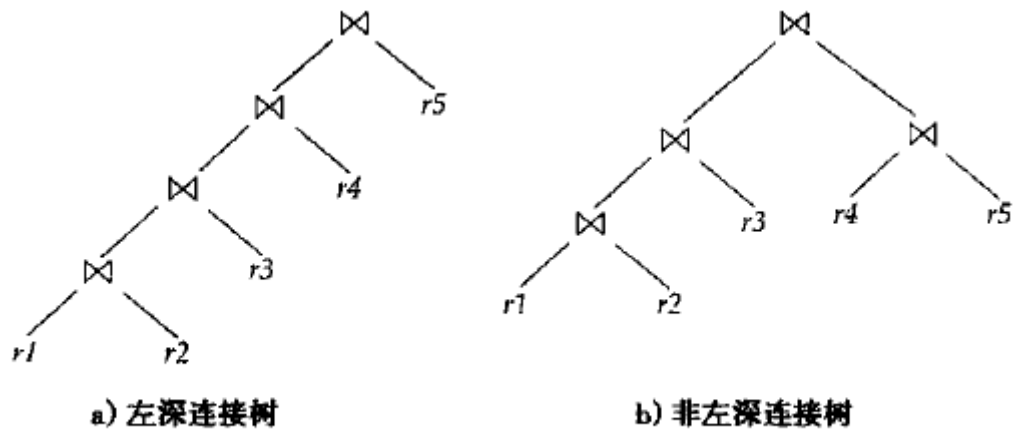


图 13-8 左深连接树

许多优化器允许为查询优化指定**优化成本预算**，当超过预算时停止搜索，返回当前找到的最优计划

13.7 总结

见P350